

情報工学科 講義 デジタル信号処理B (2026/06/18)

第3回 (Aからカウントして第10回) 微分可能信号処理

情報工学科 准教授 高道 慎之介



Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
A - 第01回	2026/04/09	イントロダクション，デジタル信号処理	フーリエ変換の基礎 (応用数学の範囲)
A - 第02回	2026/04/16	フーリエ級数展開・フーリエ変換・離散フーリエ変換	
A - 第03回	2026/04/23	ラプラス変換から z 変換へ	
A - 第04回	2026/04/30	インパルス応答と伝達関数，安定性	
A - 第05回	2026/05/07	デジタルフィルタ	フーリエ変換の 工学利用
A - 第06回	2026/05/14	高速フーリエ変換と短時間フーリエ変換	
A - 第07回	2026/05/21	総合演習．期末試験の練習としての立ち位置．	これまでの復習
期末試験	2026/06/XX	(日程は後日アナウンス)	
B - 第08回	2026/05/28	適応信号処理	現代的な信号処理を 理解するための数学
B - 第09回	2026/06/11	ベクトル・行列微分の基礎と応用	
B - 第10回	2026/06/18	微分可能信号処理	現代的な信号処理
B - 第11回	2026/06/25	グラフ信号処理入門	
B - 第12回	2026/07/02	ウェーブレット分析	
B - 第13回	2026/07/09	音声信号処理	おまけ
B - 第14回	2026/07/16	総合演習．期末試験の練習としての立ち位置．	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：ノイズキャンセリングを例に考える

- デジタル回路は次数 N の FIR フィルタとする。

フォワード計算

$$y[n] = \sum_{k=0}^{N-1} w_k[n] x[n-k]$$

最小化したい目的関数

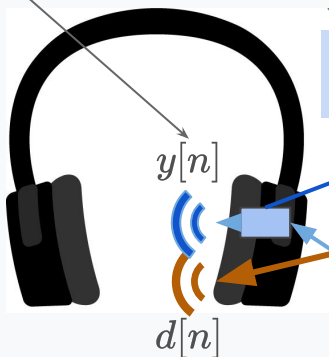
$$e^2[n] = (d[n] - y[n])^2 = \left(d[n] - \sum_{k=0}^{N-1} w_k[n] x[n-k] \right)^2$$

目的関数の値を最小化するようにフィルタ係数を求めたい

漏れる雑音を打ち消す音

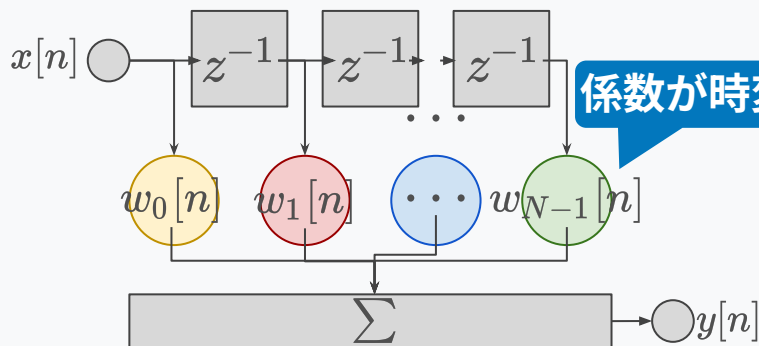
ノイズキャンセラ

デジタル回路

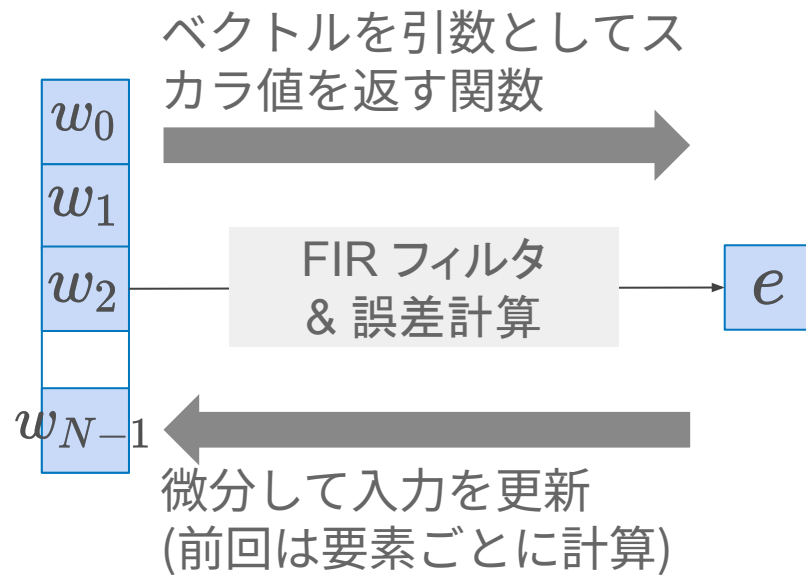


外部から
混入する雑音

内部に漏れる雑音（経路や特性は未知かつ時変）



前のページの式はこのように見ることができる



ベクトルで微分すべきなのは？
ベクトルで微分するって何？

前回の内容

スカラ値関数をスカラで微分する → スカラ

スカラ値関数をベクトルで微分する → ベクトル

スカラ値関数を行列で微分する → 行列

ベクトル値関数をベクトルで微分する → 行列

スカラー値関数をベクトルで微分する：その定義

スカラー値関数をベクトルで微分するとベクトルになる

$$\frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_d} \right]^\top$$

各次元の偏微分

例：x に関する一次式

$$f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} = \sum_i a_i x_i$$
$$\frac{\partial f}{\partial x_i} = a_i \longrightarrow \frac{\partial f}{\partial \mathbf{x}} = \mathbf{a}$$

例：x に関する二次式

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x}$$
$$\frac{\partial f}{\partial x_i} = \frac{\partial}{\partial x_i} \sum_{j=1}^d x_j^2 = 2x_i \longrightarrow \frac{\partial f}{\partial \mathbf{x}} = 2\mathbf{x}$$

スカラー関数を行列で微分する：その定義

スカラー関数を行列で微分すると行列になる

$$\frac{\partial f}{\partial \mathbf{X}} = \left[\frac{\partial f}{\partial x_{ij}} \right]_{ij}$$

行列の i, j 番目の要素

例：トレース関数

$$f(\mathbf{X}) = \text{tr}(\mathbf{A}^\top \mathbf{X})$$
$$= \sum_p \sum_q A_{pq} X_{pq}$$

例：ノルム平方

$$f(\mathbf{X}) = (\mathbf{X}\mathbf{w} - \mathbf{a})^\top (\mathbf{X}\mathbf{w} - \mathbf{a})$$

ベクトル関数をベクトルで微分する：その定義

ベクトル関数をベクトルで微分すると行列 (ヤコビアン) になる


$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \left[\frac{\partial f_i}{\partial x_j} \right]_{ij}$$

出力側が列，入力側が行になるので注意。
(スカラー値ベクトルのときは，入力側が列)

$\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ (アフィン変換，深層学習の線形層，
 $\mathbf{b}=0$ なら離散フーリエ変換, etc.)

$$f_i = \sum_r A_{ir} x_r + b_i \longrightarrow \frac{\partial f_i}{\partial x_j} = A_{ij} \longrightarrow \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \mathbf{A}$$

連鎖律 (chain rule) と計算グラフ

連鎖律 (chain rule) もスカラー値関数と同様に適用可能

$$\begin{array}{c} \boxed{} \\ x \in \mathbb{R}^N \end{array} \longrightarrow \boxed{} \quad f(x) = Ax + b \quad \boxed{} \quad y \in \mathbb{R}^M \longrightarrow \boxed{} \quad g(y) = Cy + d \quad \boxed{} \quad z \in \mathbb{R}^L$$
$$\boxed{\phantom{\frac{\partial g}{\partial x}}} \quad \frac{\partial g}{\partial x} = \frac{\partial g}{\partial f} \frac{\partial f}{\partial x} = CA \quad \boxed{} \quad \boxed{}$$

計算をつなぎ合わせた「計算グラフ」として記述可能



本日の内容

ニューラルネットワーク (NN) を知ろう (思い出そう)

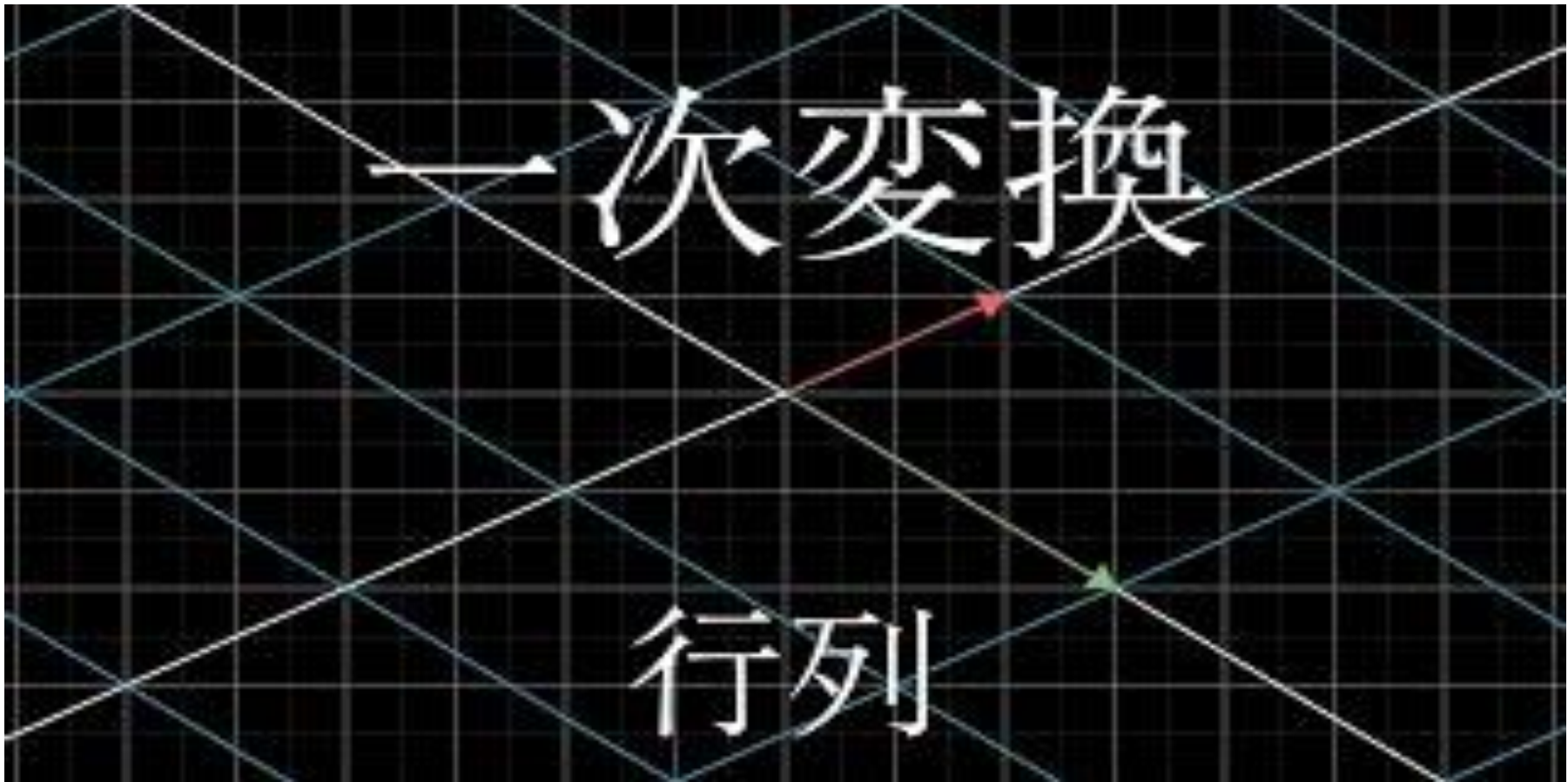
デジタル信号処理が NN と同様に記述出来ることを知ろう



信号処理と NN を併用してシステムを作れることを知ろう

ニューラルネットワークを知ろう (思い出そう)

線形変換 $y = Ax$ とアフィン変換



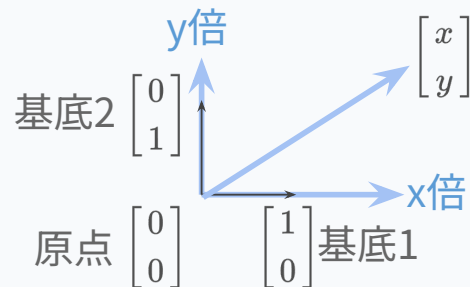
一次変換

行列

線形変換 $y = Ax$ とアフィン変換

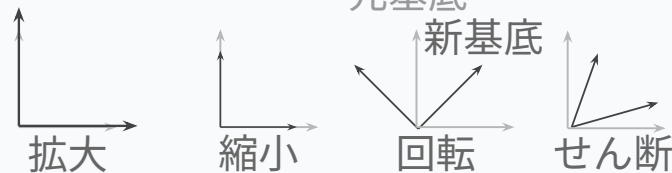
ベクトルは「基底ベクトルを何倍するか」の集まり

$$\begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} 1 \\ 0 \end{bmatrix} + y \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



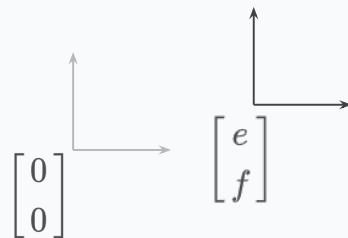
線形変換 $y = Ax$ は基底ベクトルの入れ替え．行列は新たな基底ベクトルの集まり

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix}$$



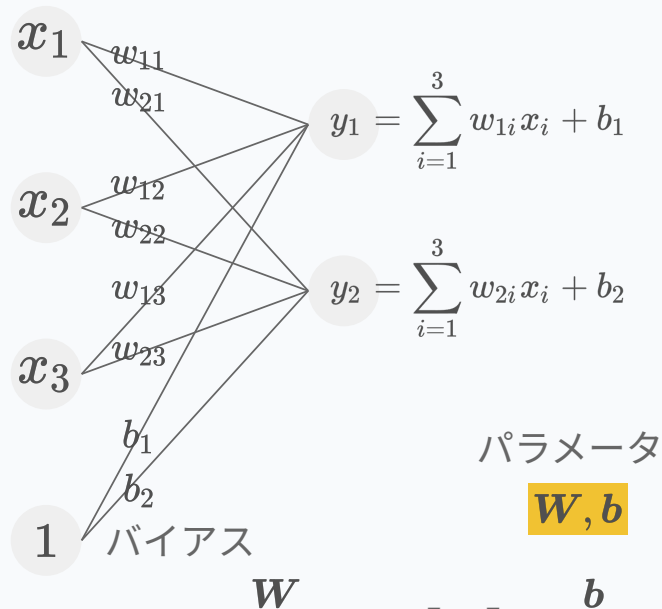
アフィン変換 $y = Ax + b$ は基底ベクトルの入れ替え&シフト

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix} = \begin{bmatrix} ax + by + e \\ cx + dy + f \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$



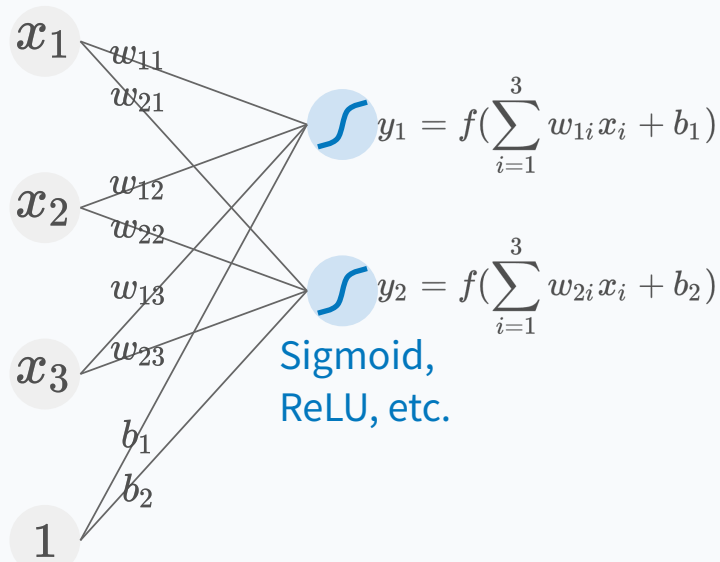
単純パーセプトロン & 活性化関数

活性化関数のない単純パーセプトロンは
アフィン変換と等価



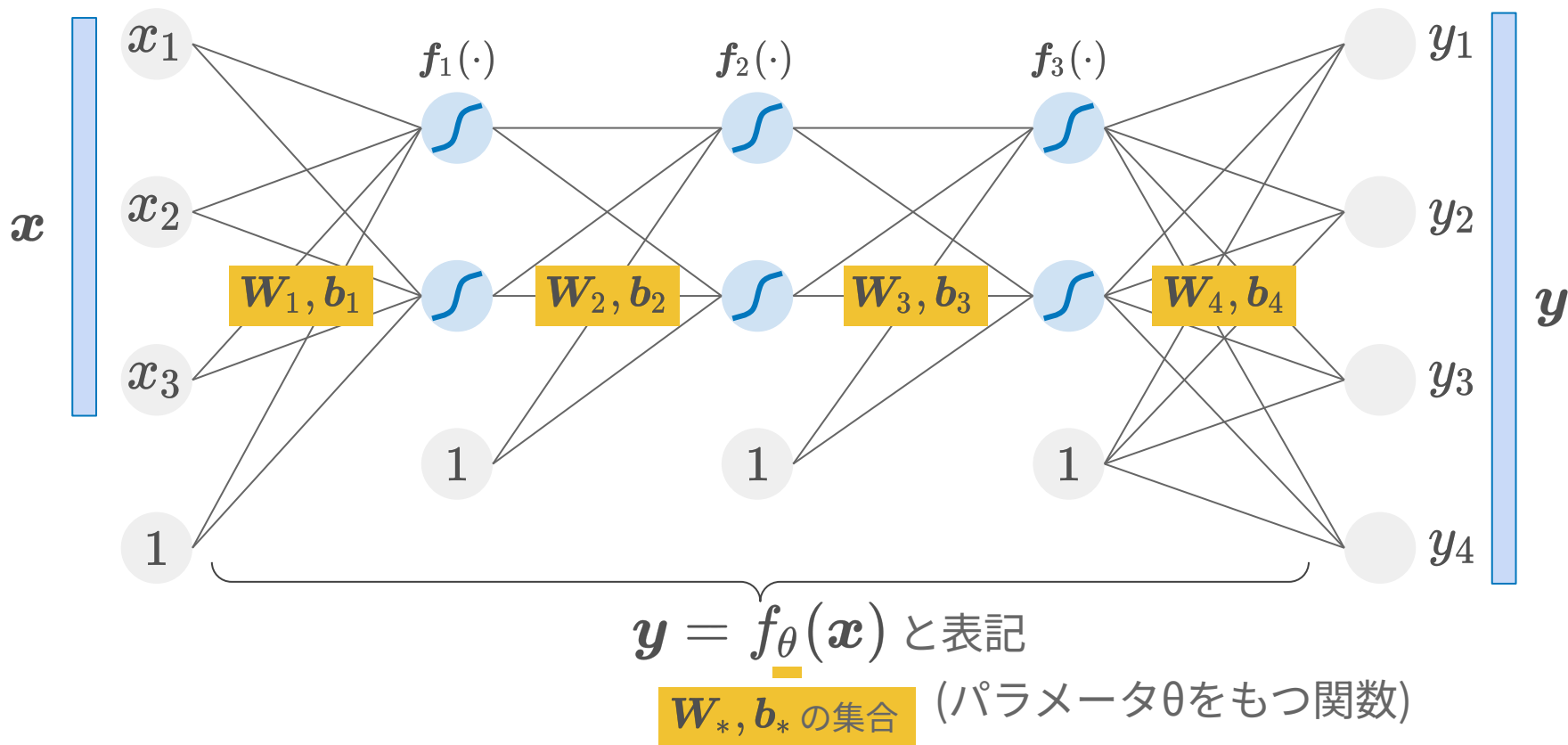
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

活性化関数により非線形変換が可能

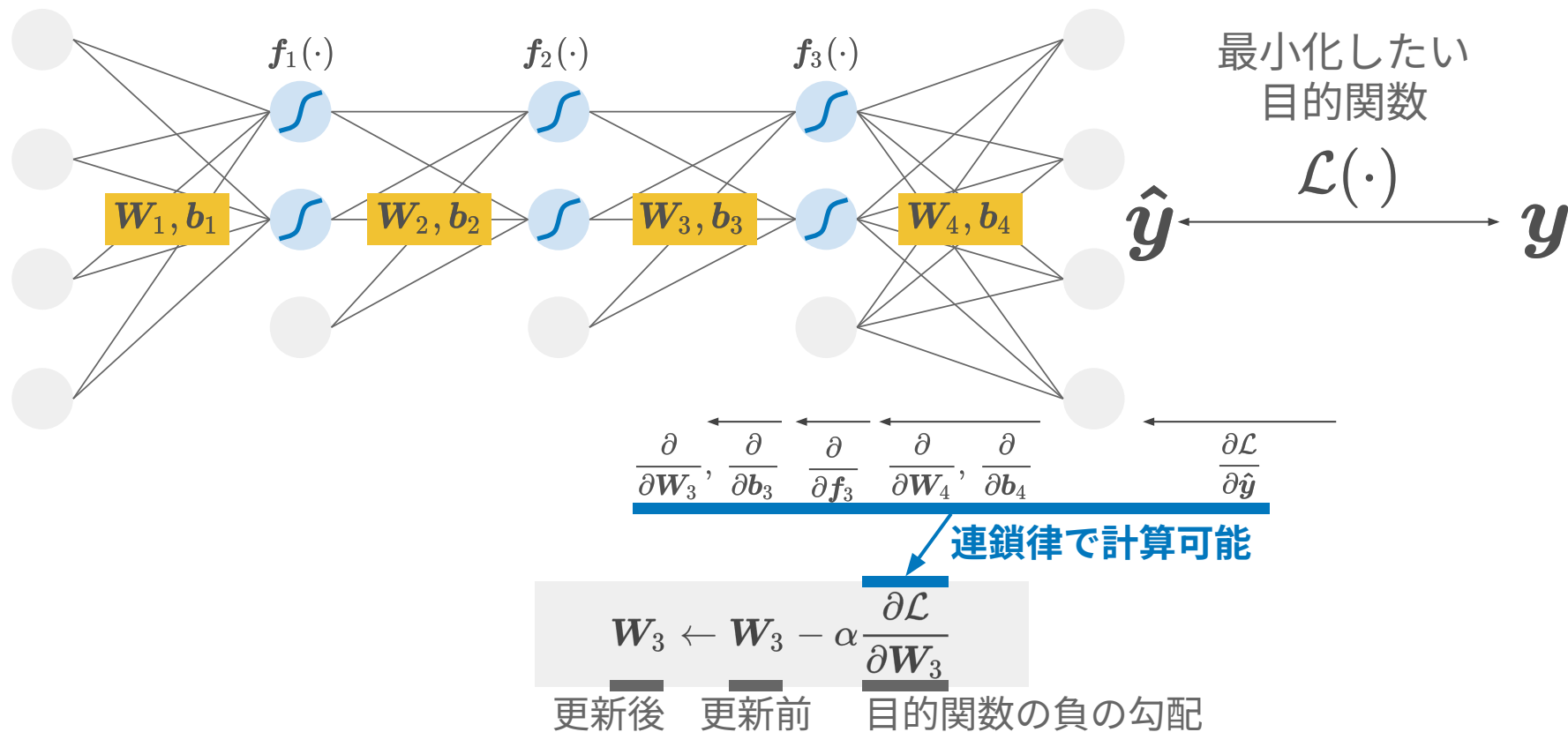


$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = f \left(\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \right)$$

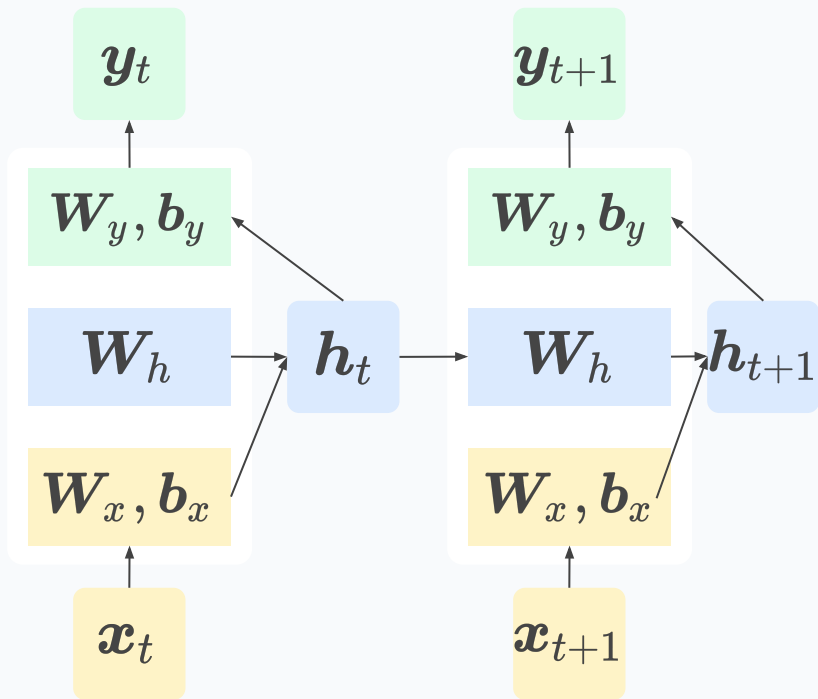
多層パーセプトロン



バックプロパゲーションによる学習



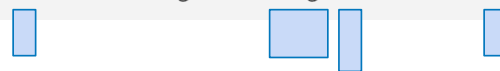
準備：リカレントニューラルネットワーク (RNN)



$$h_t = f_h (W_h h_{t-1} + W_x x_t + b_x)$$



$$y_t = f_y (W_y h_t + b_y)$$



- 1つ前の時刻の内部状態 h を使う
- 以下の3つからなる
 - 入力 x に作用する項
 - **前の時刻の h に作用する項**
 - 出力 y を計算する項

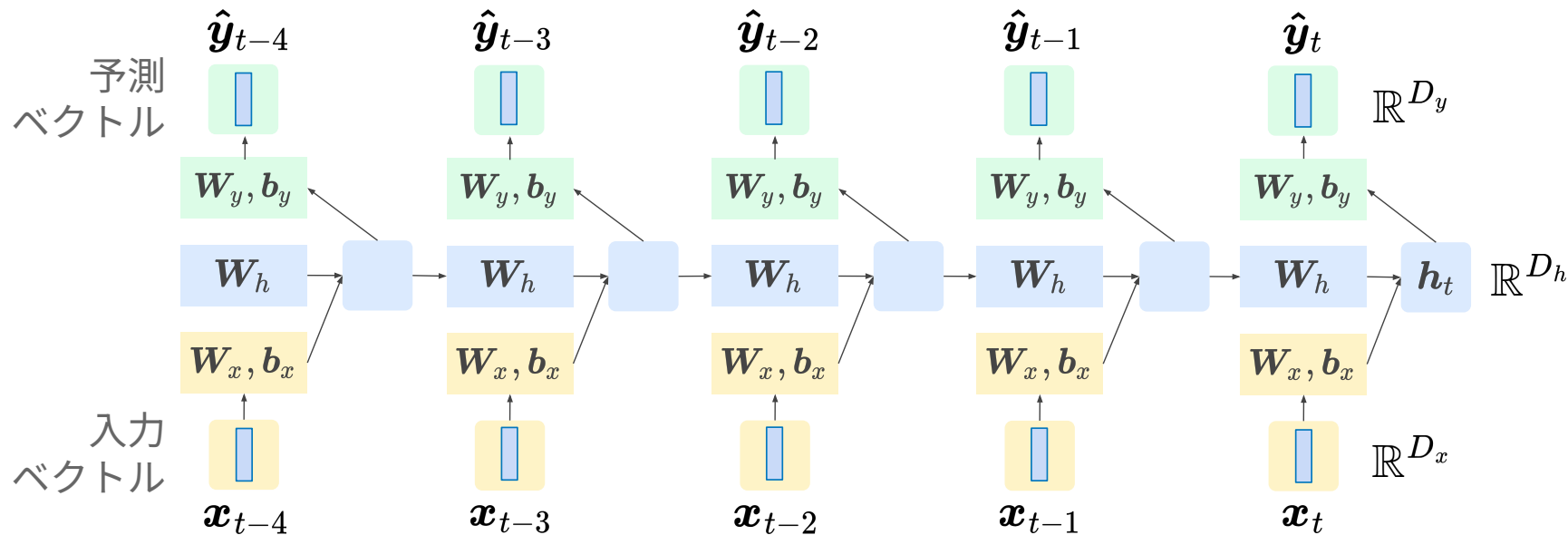


W_x, b_x

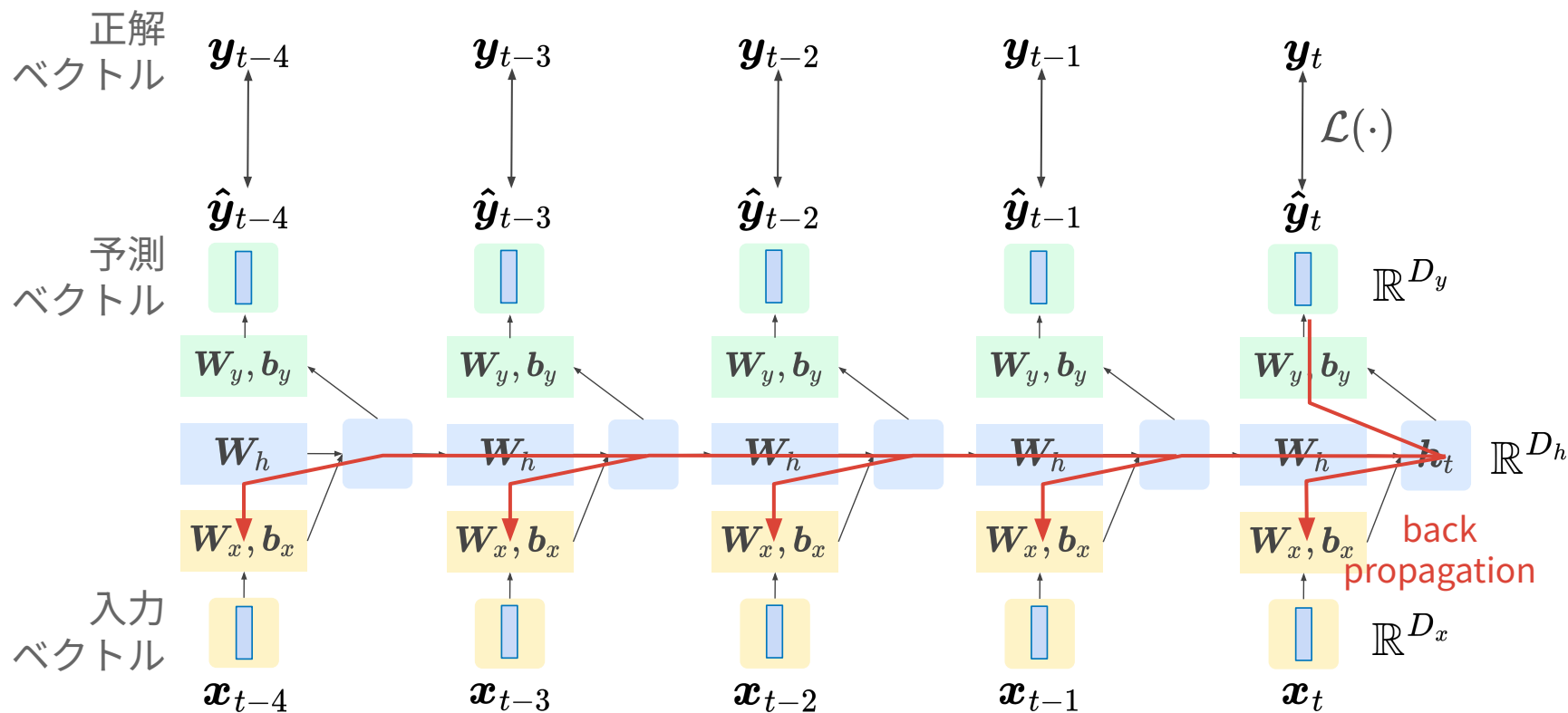
W_h

W_y, b_y

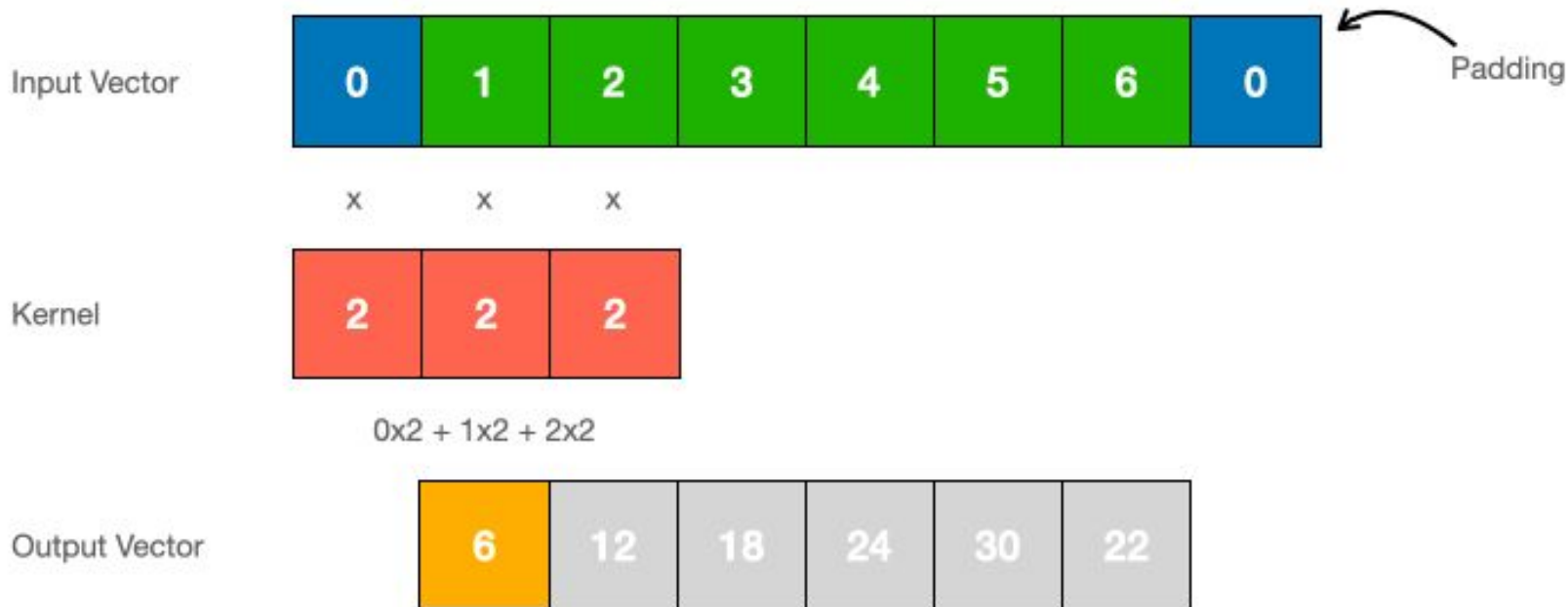
RNN による推論



RNN におけるバックプロパゲーション



畳み込みニューラルネットワーク (CNN)



$$y = h \star x$$

信号処理をニューラルネットワークと同様に記述

① 離散フーリエ変換

復習：離散フーリエ変換とその逆変換

- 離散 Fourier 変換：離散時間信号 $\rightarrow$ 離散周波数

$$X[k] = \sum_{n=0}^{N-1} x[n] \exp\left(-j\frac{2\pi kn}{N}\right) \quad \text{DFT}[x[n]] = X[k]$$

$(k \in \{0, 1, \dots, N-1\})$

- 逆離散 Fourier 変換：離散周波数 $\rightarrow$ 離散時間信号

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \exp\left(j\frac{2\pi kn}{N}\right) \quad \text{DFT}^{-1}[X[k]] = x[n]$$

$(n \in \{0, 1, \dots, N-1\})$

復習：その行列表現

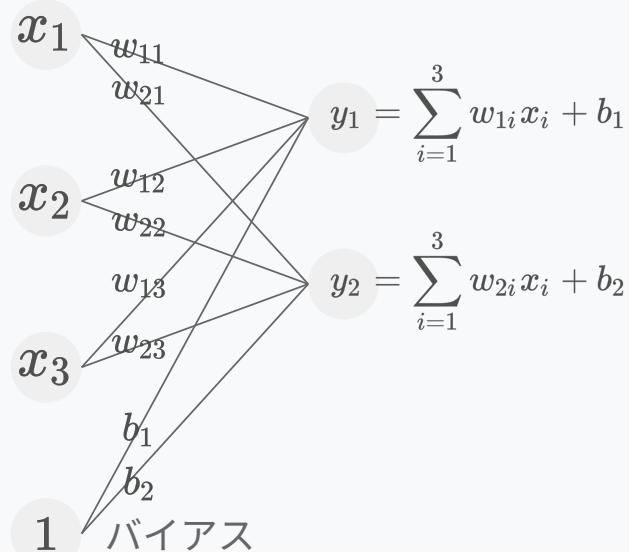
$$\mathbf{X} = \mathbf{W}_{\mathcal{F}} \mathbf{x}$$

$$\mathbf{X} \begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{array}{|c|} \hline \text{N-by-N} \\ \text{matrix } \mathbf{W} \\ \hline \end{array} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix} \mathbf{x}$$

$$(\mathbf{W}_{\mathcal{F}})_{nk} = \exp \left(-j2\pi \frac{kn}{N} \right)$$

DFTも単純パーセプトロンと同様に行列で記述できる

単純パーセプトロン



$$\underset{\mathbb{R}^{D_y}}{\mathbf{y}} = \underset{\mathbb{R}^{D_y \times D_x}}{\mathbf{W}} \underset{\mathbb{R}^{D_x}}{\mathbf{x}} + \mathbf{b}$$

離散フーリエ変換 (DFT)

$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} \text{N-by-N} \\ \text{matrix } \mathbf{W} \end{bmatrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

$$\underset{\mathbb{C}^{D_x}}{\mathbf{X}} = \underset{\mathbb{C}^{D_x \times D_x}}{\mathbf{W}_F} \underset{\mathbb{R}^{D_x}}{\mathbf{x}}$$

信号処理をニューラルネットワークと同様に記述

②振幅スペクトルと位相スペクトル

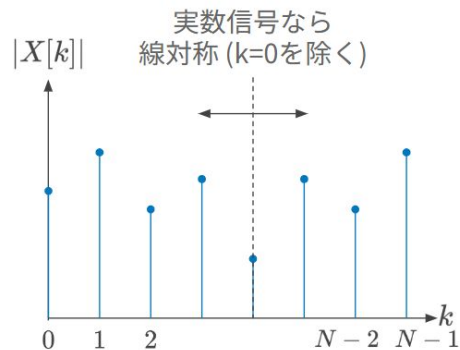
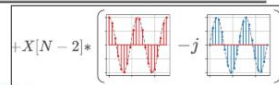
復習：振幅スペクトルと位相スペクトル

振幅スペクトル $|X[k]|$ と位相スペクトル $\arg(X[k])$

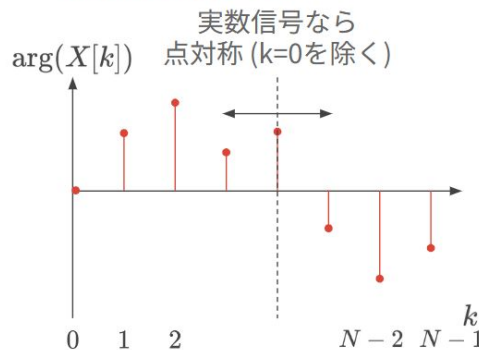
$$X[k] = |X[k]| e^{j \arg(X[k])}$$

$\exp\left(j\frac{2\pi kn}{N}\right)$ の大きさを
何倍するか

$\exp\left(j\frac{2\pi kn}{N}\right)$ の開始時刻を
どれくらい動かすか



$$\sqrt{a^2 + b^2}$$

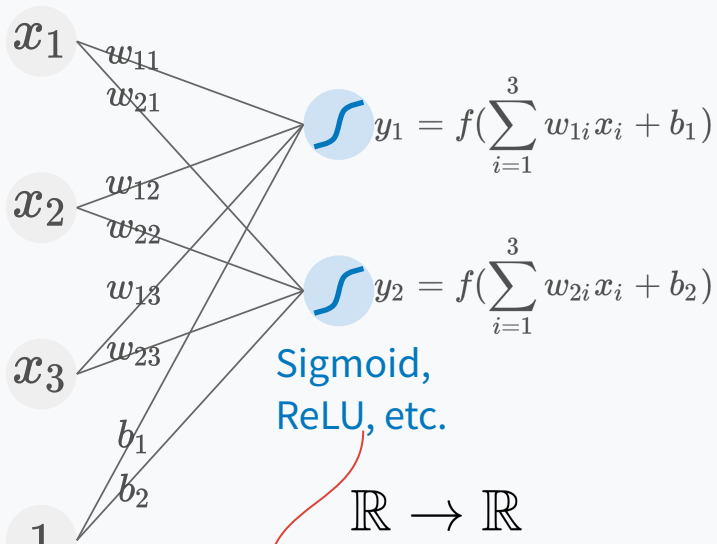


$$\tan^{-1} \frac{b}{a}$$

$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} & & \\ & \text{N-by-N} \\ & \text{matrix W} \end{bmatrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

振幅 & 位相の計算も、活性化関数と同様に非線形変換

単純パーセプトロン w/ 活性化関数



$$\mathbf{y} = \underline{\mathbf{f}}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

離散フーリエ変換 (DFT) w/ 振幅&位相

$\tan^{-1} \frac{b}{a}$

$\sqrt{a^2 + b^2}$

$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \text{N-by-N matrix } W \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$

$$\mathbb{C} \rightarrow \mathbb{R}$$

$$\mathbf{X}_{\text{abs}} = \underline{\mathbf{f}}_{\text{abs}}(\mathbf{W}_{\mathcal{F}}\mathbf{x})$$

信号処理をニューラルネットワークと同様に記述

③FIRフィルタ

復習：FIR フィルタ

FIR (finite impulse response)

差分方程式

入力にのみ依存

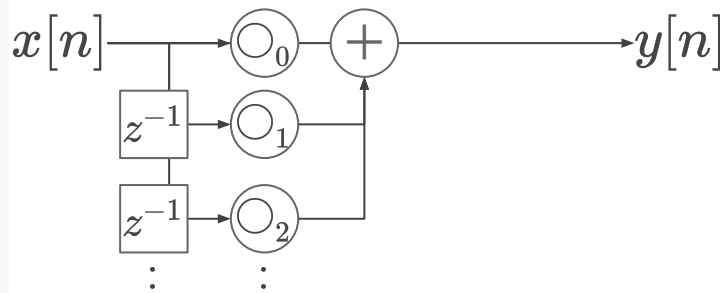
$$y[n] = \sum_m \circ_m x[n - m]$$

伝達関数

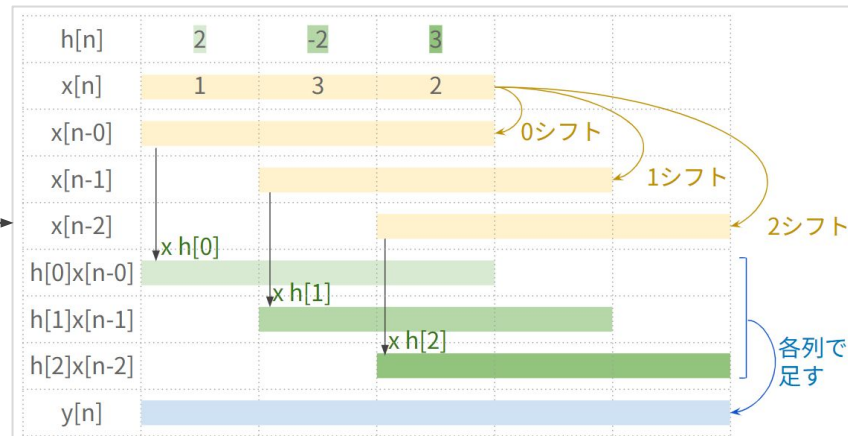
分子のみ

$$H(z) = \sum_m \circ_m z^{-m}$$

ブロック図



要は畳み込み演算



FIR フィルタと CNN は似ている…？

FIR (finite impulse response)

差分方程式

入力にのみ依存

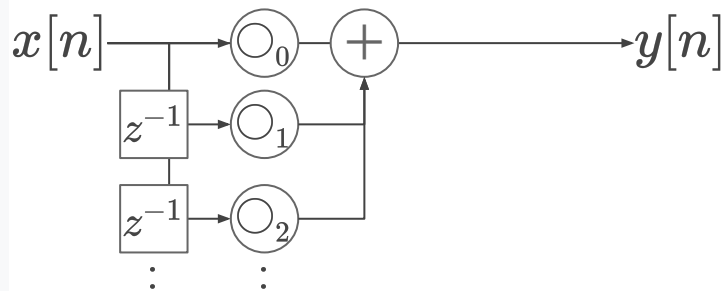
$$y[n] = \sum_m \circ_m x[n - m]$$

伝達関数

分子のみ

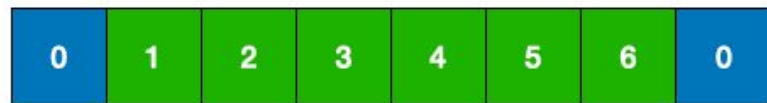
$$H(z) = \sum_m \circ_m z^{-m}$$

ブロック図

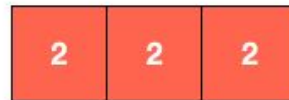


フィードフォワードのみ

CNN (convolutional neural network)



x x x



$0 \times 2 + 1 \times 2 + 2 \times 2$



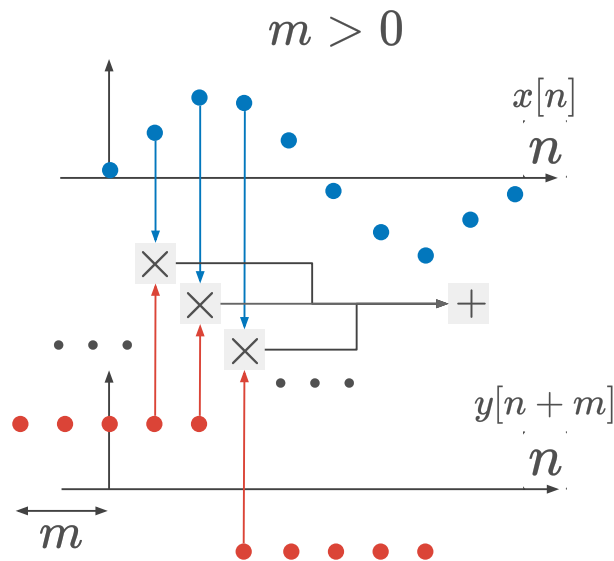
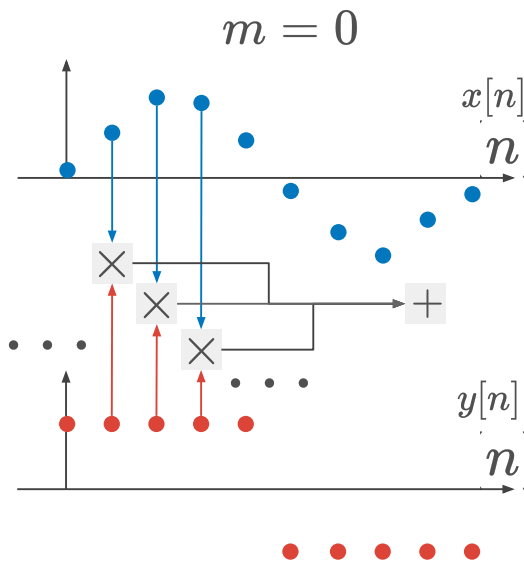
こちらも“畳み込み”だがFIRと等価でない
→ **実は、CNNの畳み込みは相互相関関数！**

復習：相互相関関数の定義

相互相関関数

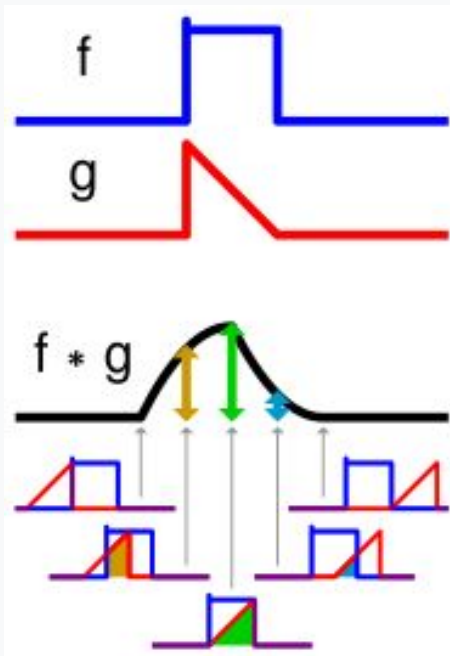
$$\phi_{xy}[m] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]y[n+m]$$

- 2つの異なる信号の一致度を測る関数.
- 片方の信号に対してずらした時間 (m) を引数にして、信号間の一致度 (積の総和) を返す.

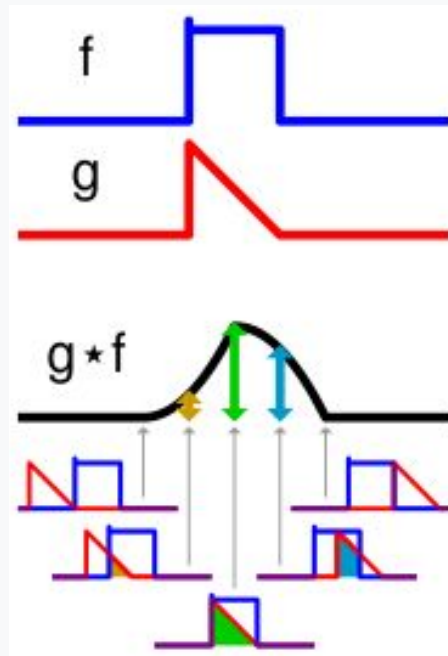


時間反転したフィルタの畳み込みは相互相関と等価

FIR の畳み込み



CNN の畳み込み



FIRフィルタを時間反転して相互相関を計算するとCNNの畳み込み結果に一致。
畳み込みと相互相関の定義式から証明可能。

FIR フィルタのニューラルネットワーク的表現

FIR (finite impulse response)

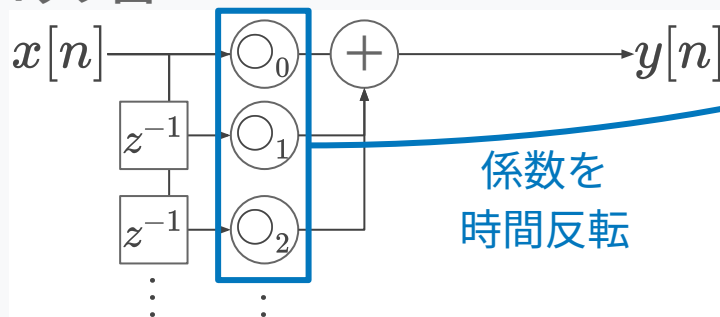
差分方程式

$$y[n] = \sum_m \circ_m x[n - m]$$

伝達関数

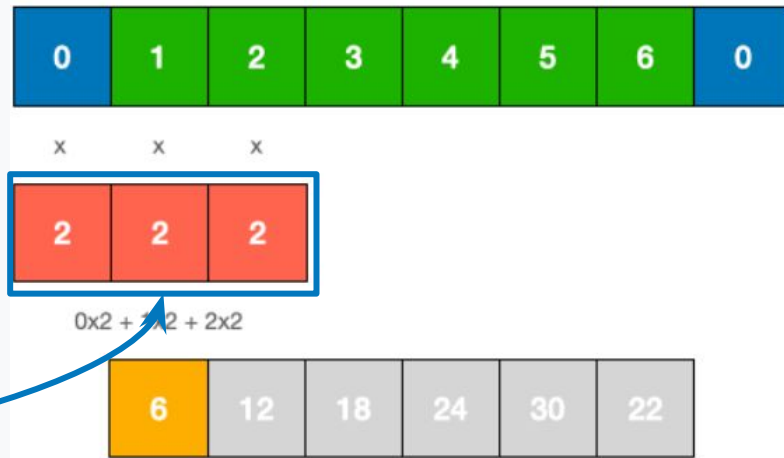
$$H(z) = \sum_m \circ_m z^{-m}$$

ブロック図



係数を
時間反転

CNN (convolutional neural network)



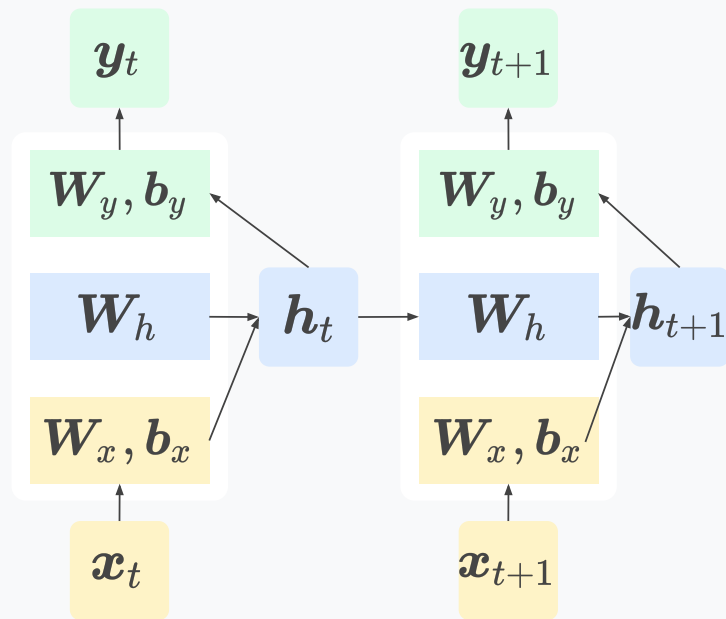
フィルタ係数を時間反転させると、
FIR フィルタと CNN は等価

信号処理をニューラルネットワークと同様に記述

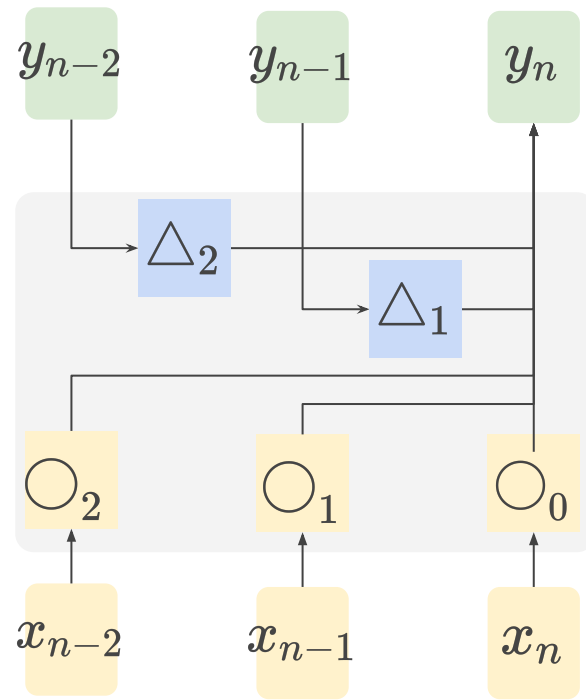
④IIRフィルタ

RNN と比較する

RNN (recurrent neural network)



出力項が恒等写像，次数 1 なら IIR と等価．
x と y それぞれに対する CNN でも記述可能



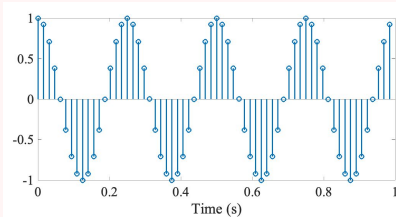
信号処理をニューラルネットワークと同様に記述

⑤窓関数と短時間フーリエ変換

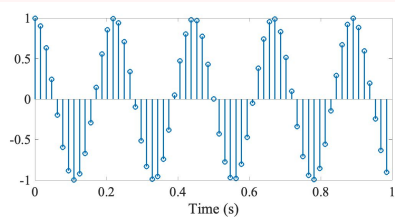
復習：離散Fourier変換の問題

信号長が 1 sec (左) の場合 vs. 信号長が 2 sec (右) の場合

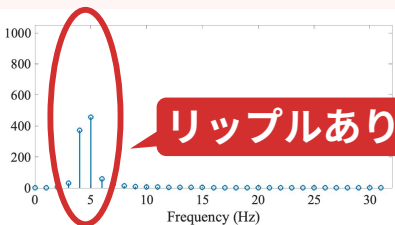
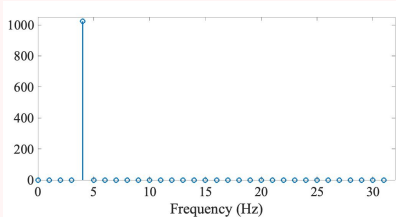
信号1 (4 Hz)



信号2 (4.5 Hz)

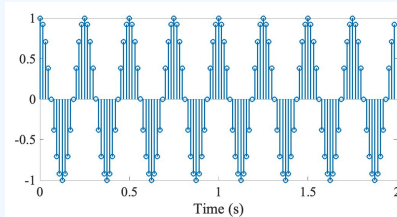


DFT & $|\cdot|^2$ (パワースペクトル)

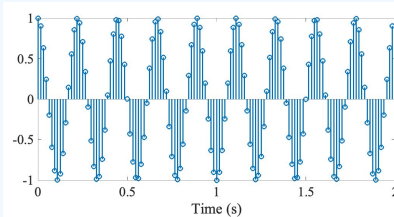


周波数 4.5 Hz の近傍にスペクトルが
染み出してしまっている…

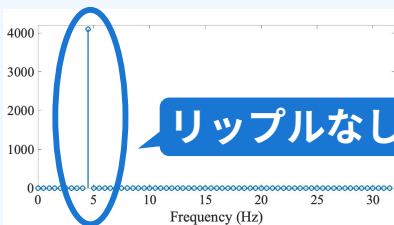
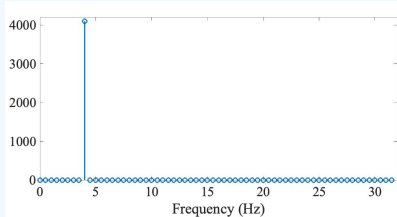
信号1 (4 Hz)



信号2 (4.5 Hz)



DFT & $|\cdot|^2$ (パワースペクトル)

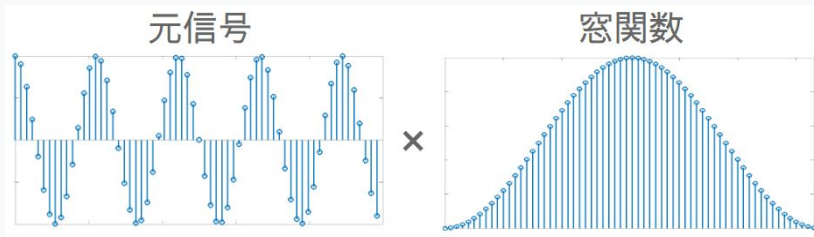


同じ信号のはずなのに、この場合は
染み出していない．何故？

復習：窓関数の役割

窓関数によるリップルの抑圧

信号長を対象に合わせて変更することは通常できないため、両端のつながりの影響を少なくする窓関数を信号に乗算してリップルを抑制する。



窓関数の乗算

離散窓 Fourier 変換

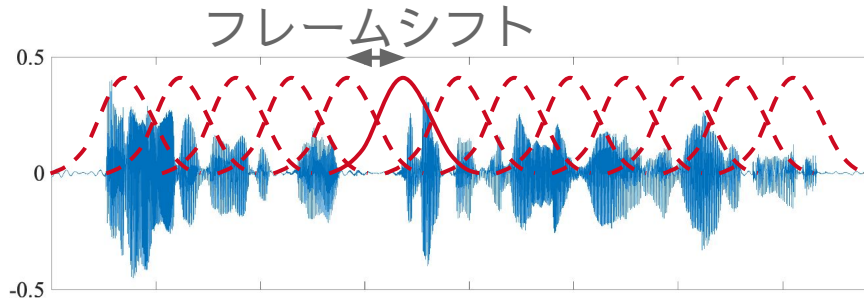
離散時間信号 $x[n]$ に対し, 信号区間の中心から離れるほど減衰する重み関数 (窓関数) $w[n]$ を乗じて離散 Fourier 変換

$$X[k] = \sum_{n=0}^{N-1} \underline{w[n]x[n]} \exp\left(-j\frac{2\pi kn}{N}\right)$$

窓関数の乗算

復習：短時間 Fourier 変換：時間変化する信号の分析

- 時々刻々と変化する時系列信号を考える。
 - 長い時間区間の信号では、スペクトルの時間変化に意味がある場合が多い
 - (時間区間全体のスペクトルはあまり意味を持たない)
- 短時間 Fourier 変換 (short-time Fourier transform: STFT)
 - 短時間の時間区間ごとに窓関数を乗じて Fourier 変換

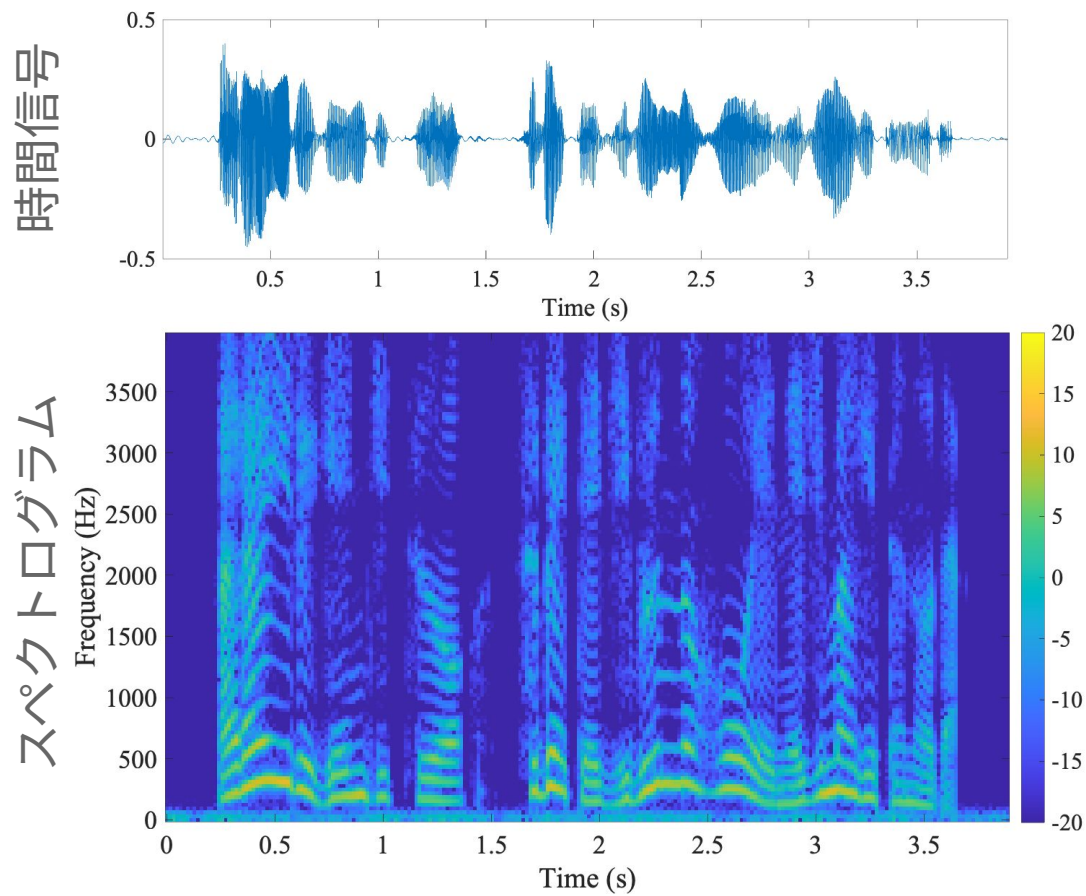


- 連続系での数式表現 (離散の場合も基本的に同じ)

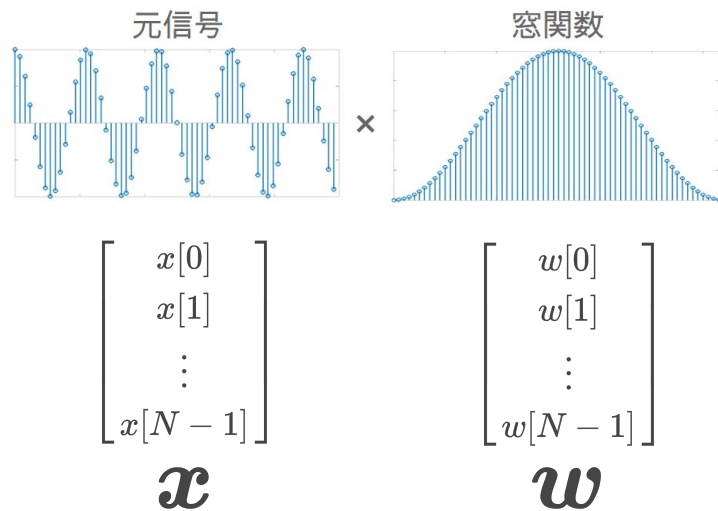
$$X(\omega, t) = \int_{-\infty}^{\infty} x(\tau) \underbrace{w(\tau - t)}_{\text{時刻 } t \text{ を中心とした窓関数}} \exp(-j\omega\tau) d\tau$$

時刻 t を中心とした窓関数

復習：スペクトログラム

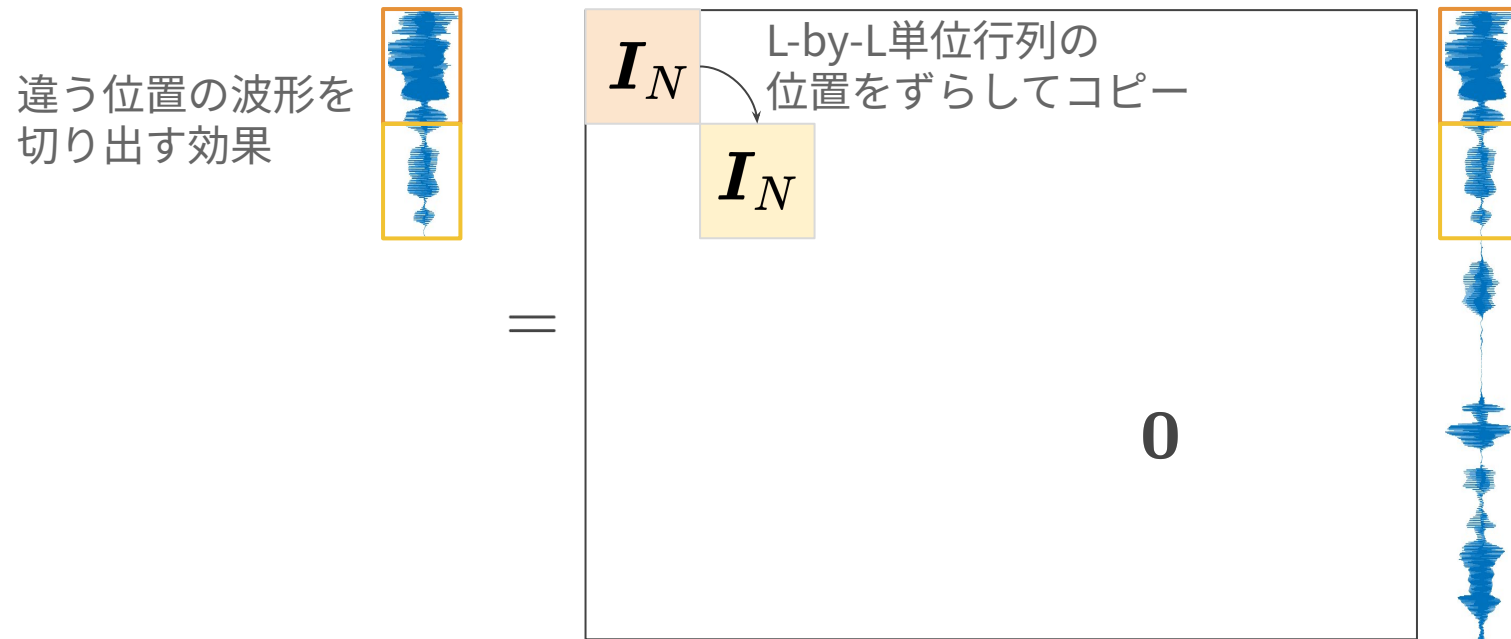


窓かけはアダマール積 (要素積)



$$\mathbf{x} \odot \mathbf{w} = \text{diag}[\mathbf{w}] \cdot \mathbf{x} = \begin{bmatrix} x[0]w[0] \\ x[1]w[1] \\ \vdots \\ x[N-1]w[N-1] \end{bmatrix}$$

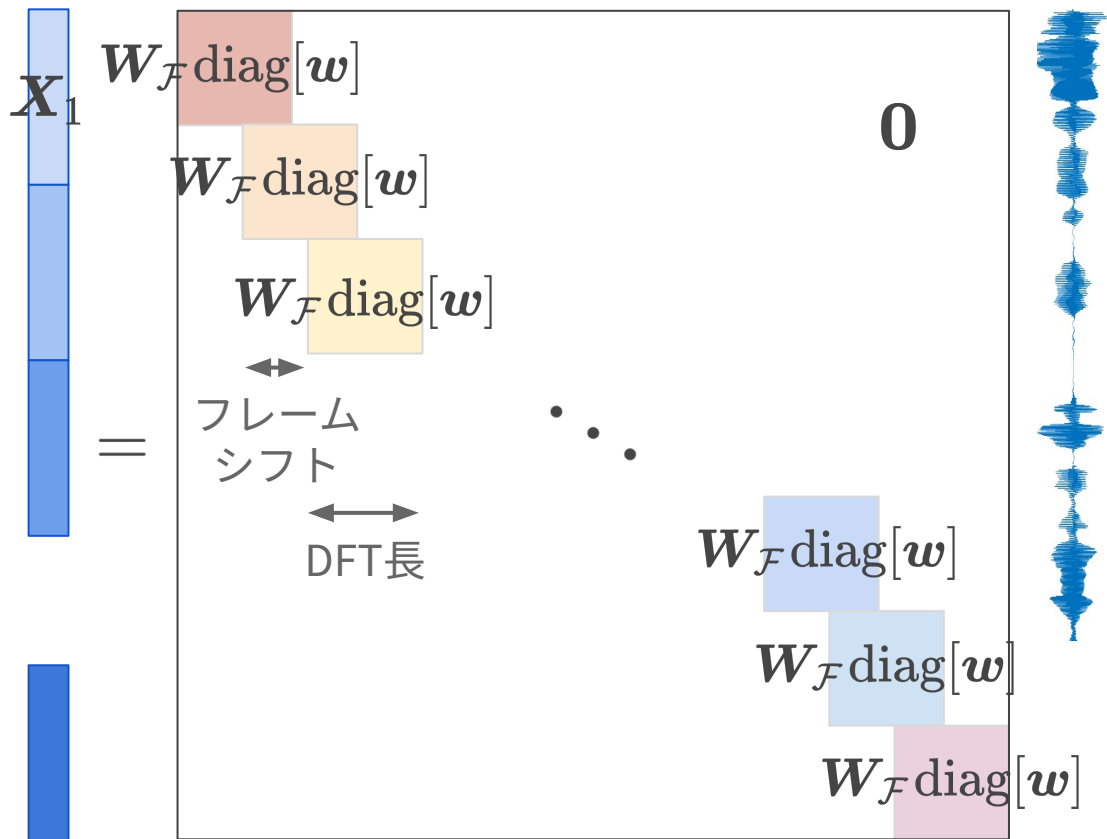

時間シフトは部分行列のシフト



短時間フーリエ変換の行列表現

(=ニューラルネットワークと同様に記述可能)

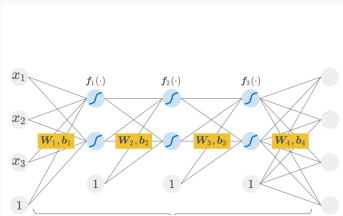
1フレーム目に対する変換結果



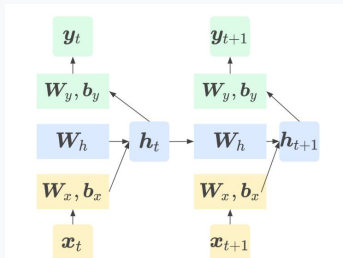
信号処理とニューラルネットワークは1つのシステムとして記述できる

本スライドで触れた内容は全て同様に記述可能

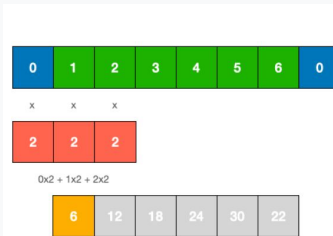
多層パーセプトロン (MLP)



リカレント NN (RNN)



畳み込み NN (CNN)



離散フーリエ変換 (DFT)

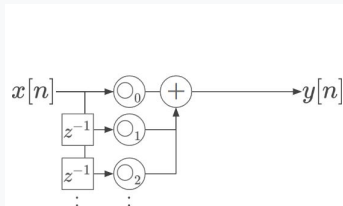
$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} \text{N-by-N} \\ \text{matrix W} \end{bmatrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

振幅/位相スペクトル

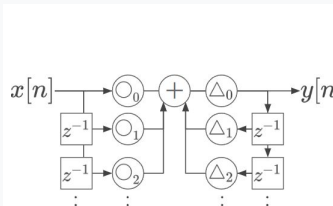
Diagram illustrating the mapping of trigonometric functions to vector components:

- The expression $\tan^{-1} \frac{b}{a}$ (pink box) maps to the angle component.
- The expression $\sqrt{a^2 + b^2}$ (blue box) maps to the magnitude component.
- These components are used to define the vector X , specifically the element $X[k]$.

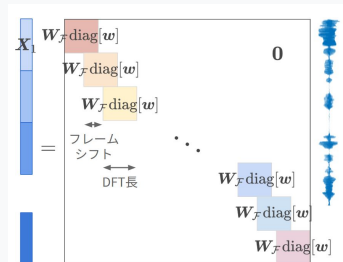
FIR フィルタ



IIR フィルタ



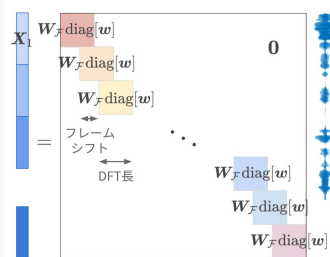
短時間フーリエ変換 (STFT)



併用して1つのシステムを作ることが可能

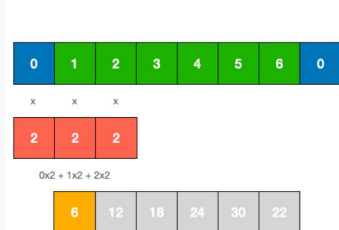


STFT



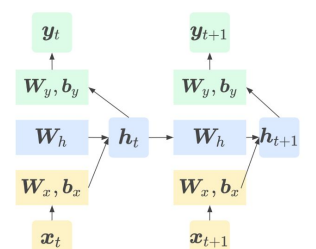
時間周波数変換

CNN



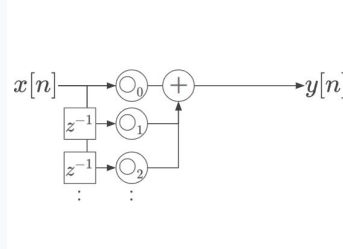
近傍の時間周波数の値を集計

RNN



過去の計算結果を現在の予測に利用

FIR



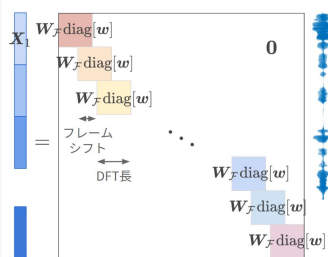
過去の計算結果を現在の予測に利用

$\hat{y}$

バックプロパゲーションも可能

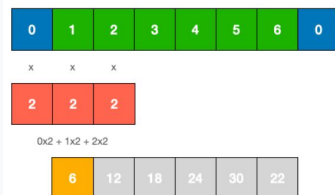


STFT



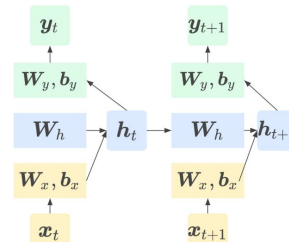
時間周波数変換

CNN



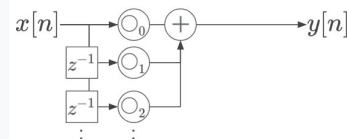
近傍の時間周波数の値を集計

RNN



過去の計算結果を現在の予測に利用

FIR



過去の計算結果を現在の予測に利用

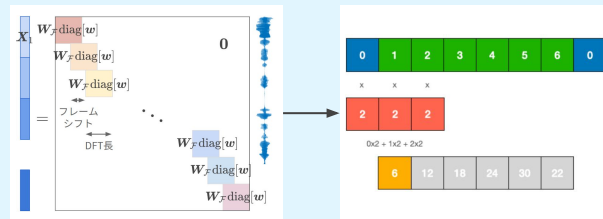
$$\hat{y} \leftrightarrow y$$
$$\mathcal{L}(\cdot)$$

勾配を使ってパラメータを更新
(途中に信号処理があっても可能)

併用するメリット

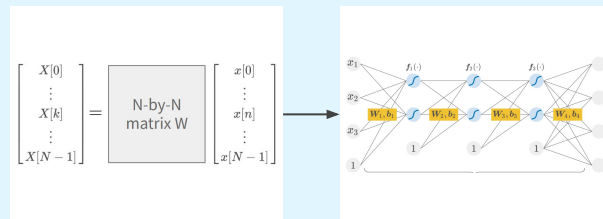
信号処理の理論とNN のデータ駆動を併せ持つ

例えば STFT で時間周波数特徴を抽出でき、
その後のパラメータは教師データに基づいて最適化



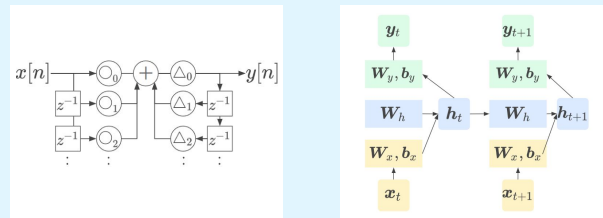
信号処理に基づいて NN を初期化できる

例えば MLP のパラメータを DFT 行列で初期化する。
乱数初期化に比べ、最初から周波数成分抽出を保証。



信号処理の理論を使ってシステムを評価できる

例えば RNN を IIR フィルタのように考えて「いかなる入力に対しても値が発散しないか」を評価可能



プログラミングしてみよう

jax/flax というライブラリを使います

<https://github.com/google/flax>



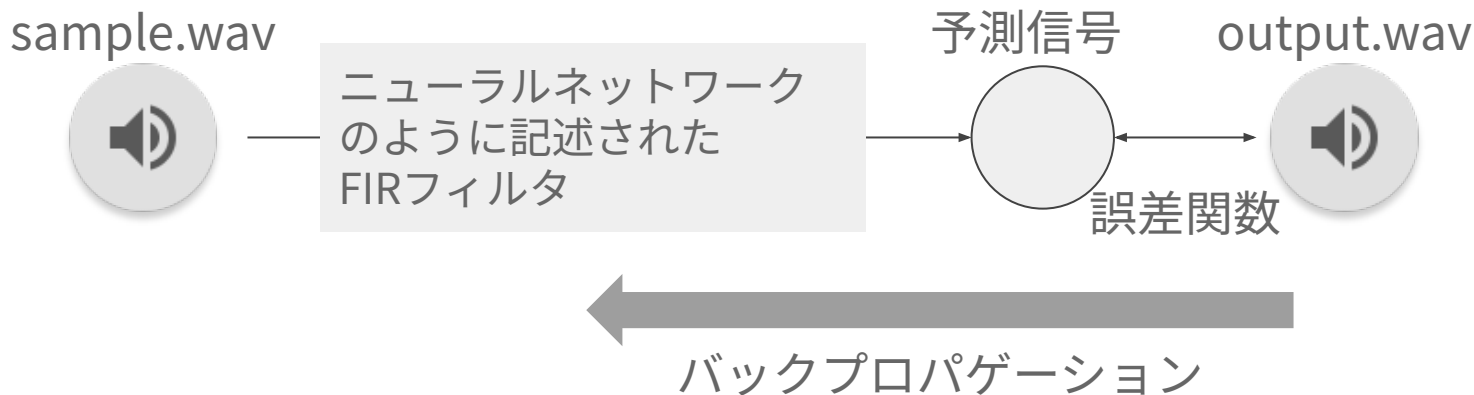
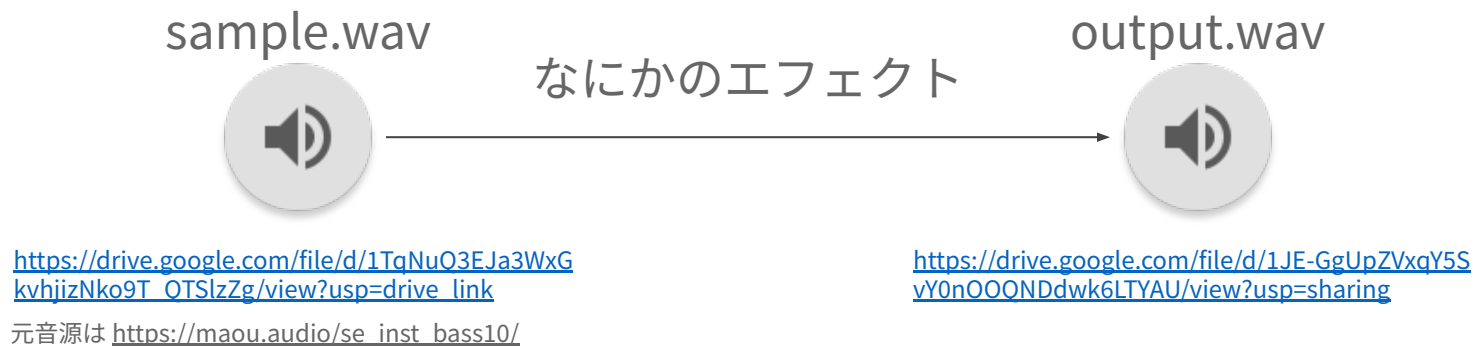
Flax: A neural network library and ecosystem for JAX designed for flexibility

JAXは非常にざっくりいうと、**機械学習用に設計されたNumPy**と説明することができます。実際、JAXの数値関数用のAPIは、NumPyと高度に互換性があり、NumPyと基本的には全く同じようにコードを実装することが可能です。このNumPy APIに加えて、機械学習に役立つ拡張として、次のような特徴があります。

JAXの使い方を紹介してきましたが、実はこのままでは深層学習の処理を直接実行すること簡単ではありません。そのために、**JAXをベースとした機械学習ライブラリを組み合わせる**必要があります。JAXを基盤とした機械学習ライブラリは複数ありますが、**Flax**はその中でも最も人気があるフレームワークの一つです。

https://zenn.dev/turing_motors/articles/4a7e80c4a47b65

演習：ベース音に掛けられたエフェクトの パラメータを求めよう



Google colab で説明します

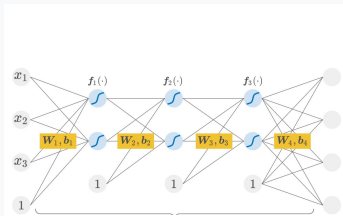
https://colab.research.google.com/drive/1sw1ykK_6lXHjudxF-N4pZvBClwOllik7?usp=sharing

自分で動かしたいときは，自分のドライブにコピーして，前のページの音ファイルをアップロードすること．

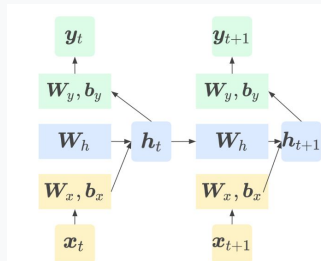
まとめ
(matome)

まとめ

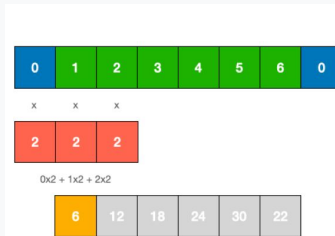
多層パーセプトロン (MLP)



リカレント NN (RNN)



畳み込み NN (CNN)



離散フーリエ変換 (DFT)

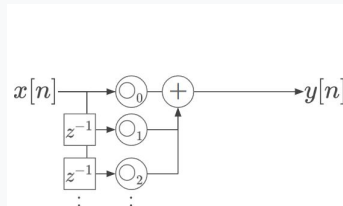
$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} \text{N-by-N} \\ \text{matrix } W \end{bmatrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

同じ枠組みで記述可能！併用したシステムも作ってよい！

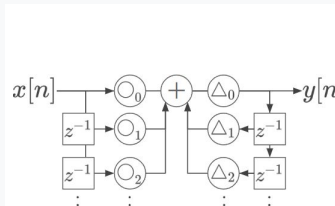
振幅/位相スペクトル

$$\begin{aligned} \tan^{-1} \frac{b}{a} &\rightarrow \begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} \\ \sqrt{(a^2 + b^2)} &\rightarrow \begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} \end{aligned}$$

FIR フィルタ



IIR フィルタ



短時間フーリエ変換 (STFT)

$$X_1 = \begin{bmatrix} W_{\mathcal{F}} \text{diag}[w] \\ W_{\mathcal{F}} \text{diag}[w] \\ W_{\mathcal{F}} \text{diag}[w] \\ \vdots \\ W_{\mathcal{F}} \text{diag}[w] \\ W_{\mathcal{F}} \text{diag}[w] \end{bmatrix} = \begin{bmatrix} \text{フレーム} \\ \text{シフト} \\ \text{DFT 長} \end{bmatrix}$$

課題：どれか1つを実施せよ．いずれも点数は同じ．

1. 簡単コース

- a. google colab の例を参考に，別の FIR フィルタを設計しエフェクトをかけよ．そのフィルタを学習で推定できるかを確認し，その学習や収束の様子を考察せよ．

2. 普通コース

- a. IIR フィルタを設計しエフェクトをかけよ．以下，同上．

3. 難しいコース

- a. <https://intro2ddsp.github.io/intro.html> を参考に，微分可能信号処理システムを構築せよ．以下，同上．